ECE 49595 SD I
Verification & Validation
Team 1
Spring 2026

GenAI Statement: We as a team did not use GenAI for this assignment, other than to generate potential ideas of topic discussion points in our assignment. Else, everything was written by us.

Objectives

The specific objectives that we intend to verify and validate are the following:

1. Verify that the LLM produces job recommendations that have at least an 80% match to inputted skills (SYS-001).
2. Verify that the job scraper captures at least 95% of any job's description (SYS-002).
3. Verify that the algorithm accurately identifies and displays the specific missing skills of a user to a specific job, in at least 90% of the cases (SYS-003).
4. Verify that the resume takes less than 2 seconds to upload, and that in 90% of the cases, analyzing a resume takes less than 20 seconds (SYS-004).
5. Verify that at least 80% of users believe that the UI/UX design of JobRec's platform is intuitive, and easy to navigate (SYS-005).

Prioritization Table

Product Capability	**Priority** (High, Medium, Low)	**Brief Justification**
2	High	The quality of job recommendations is directly related to the quality of the listing database, so it must reflect reality and be easy to parse.
1	High	Job recommendations are the largest draw of our platform, so their accuracy must be assured above other features.
5	Medium	The quality of our features is rendered useless if users cannot access them, which makes intuitive design important, if less so than the features themselves.
4	Medium	Users will expect resume analysis to take a small amount of time. If that is not maintained, they may suspect that the process has failed and stop using the system. However, this is less important than users being able to intuitively find features.
3	Low	Skill gap analysis is one of our less important features since it can easily be intuited by the user and other metrics we provide to them. However, it is still good for quality of life and would likely be easy to implement alongside high quality listing data.

Full RVTM Matrix

Our requirements full verification and traceability matrix (RVTM) is attached with our Brightspace submission for this assignment.

Test Approach

This section details our overall testing strategy for JobRec. The following section discusses unit, integration, and system testing, as well as non-function testing strategies, and the regression testing approach that we plan to implement. We end this section with testing risks that could occur when undergoing this process. Our following approach is ultimately well aligned with the critical system requirements that we have defined in the RVTM, and it focuses on validating core user flows for our platform, which includes uploading a resume, resume skill parsing, and the job recommendation algorithm.

Unit Testing

Within the test approach overall, unit testing will focus on testing and validating individual components by themselves to make sure that there is correctness. The components tested include (1) resume parsing functions (such as PDF/DOCX extraction), (2) skill extraction and normalization logic, (3) job scraping and parsing functions, and (3) the recommendation scoring logic. The approach to test these functions include using pytest, a Python tool to test backend python functions, provide controlled inputs such as known resumes/job descriptions to use, and validate the outputs of the algorithm against the expected results to see if the skill extraction was correct. The frequency as to which we will be uploading these tests would be after every code update, so that correctness of the updates is ensured before the new algorithm is added to the platform.

Integration Testing

Integration testing, regarding our platform, will validate interactions between the various subsystems in our platform, to make sure that every "piece" is working together, and that a correct data flow is going on. The specific key integrations that will be tested would be (1) the flow of a resume being uploaded, parsed, and having its key information being stored in a database, (2) a job scraper scraping information from the internet to storing it in a database to giving data to a recommendation engine, (3) taking the user's stored profile data, to passing it to the recommendation system, to displaying those results to the designed user interface. The method to test these flows is to use API-level testing through Flask endpoints to simulate user interactions, use mock/controlled datasets when needed, and validating that the outputs produced from one subsystem are correctly used by the next. An example of this process would be uploading a resume, then verifying if the extracted skills were stored properly, to verifying that the job recommendations reflect the user's extracted skills.

System Testing

System testing within JobRec will entail validating the entire end-to-end workflow against defined system requirements. The key user flows tested for this part would be (1) uploading a

resume, analyzing it, and receiving job recommendations, (2) viewing skill gaps for recommended jobs, and (3) modifying the user profile based on the updated recommendations. The approach to this would be through carrying out a full workflow using sample inputs that are realistic, using test cases which have been further defined in the RVTM (Such as TC-004, and TC-006), and validating both the correctness and performance requirements that have been defined. Ultimately, the goal is to ensure that the system fulfills all of the functional requirements defined under actual usage conditions by an actual user.

Manual Versus Automated Testing

In order to ensure that there is both efficiency and usability validation, we will use a combination of both automated and manual testing. Within automated testing, we will be using unit testing using Pytest, API endpoint testing for any backend routes, performance measurements (Such as the time it takes to upload a resume and analyze it), and regression testing using test cases as defined above. For manual testing, there will be human usability testing via surveys on UI/UX designs, in areas such as navigation, layout, clarity, etc, end-to-end workflow validation from a user perspective, edge case testing on incomplete data, or unlikely cases.

Non-Functional Testing Strategy

The non-functional requirements will be tested through numerical metrics that are aligned with system objectives. Performance will be numerically evaluated through ensuring that at least 90% of the time, resume uploads are complete in under two seconds, and resume analysis uploads are complete in under 20 seconds. Scalability will be tested through simulating multiple users using the system at the same time, and evaluating the system's capability of handling this. Reliability will be verified through checking if the system is capable of not crashing when receiving invalid or corrupted inputs. Security will be checked through ensuring that user data does not bleed to other accounts and is isolated, and checking authentication methods for protected actions. Finally, usability will be tested through user surveys of test subjects using our platform, with the goal that at least 80% of the test users find the system easy to navigate.

Regression Testing Approach

Regression testing will be conducted to make sure that the existing functionality on the platform is not broken due to new updates to the platform. This will be done by running the same test cases from the RVTM after every code update. Important system flows which include resume upload and parsing, job scraping, the job recommendation algorithm, and the skill gap detection will constantly be tested, as any bugs could be a major issue to the usage of our platform. These tests will be executed before any large milestones that need to be completed and will utilize PyTest.

Validation Plan

We will gather user feedback and usability/performance metrics three times throughout the course of the fall semester, conducting a verification test when we reach a milestone as listed in our Gantt chart.

User Acceptance Testing Approach

The goal for JobRec is to help users find relevant jobs catered to them and their skillset. We will verify this goal is met through multiple steps of verification.

We want users to provide feedback themselves as they are important stakeholders. As such, when we hit our verification/validation milestone within our Gantt chart, we will look for user feedback. Our goal is to get feedback from a minimum of 5 stakeholders on each milestone by asking random college students at Purdue to perform simple tasks as defined by our key user flows; this includes uploading their resume, navigating job recommendations, and reviewing skill gaps, unsupervised. After the user completes these tasks, we will provide them with a survey. This survey will include qualitative questions such as:

- Do the recommendations make sense?
- Does the UI feel intuitive?
- Did you feel confused at any step of the process?

And quantitative questions such as:

- Rate your satisfaction with the navigation on a scale of 1-5.
- Rate your satisfaction with the recommendations on a scale of 1-5.
- How likely would you use this website? (1-highly unlikely 5-highly likely)

Based on user feedback, we will update the system accordingly. For our validation to be considered satisfactory, we define the following success metrics based on the previous questions:

Question	**Success?**
Do the recommendations make sense?	80% of users state “Yes”.
Does the UI feel intuitive?	80% of users state “Yes”.

Did you feel confused at any step of the process?	80% of users state “No”.
Rate your satisfaction with the navigation on a scale of 1-5.	Mean >=3.5.
Rate your satisfaction with the recommendations on a scale of 1-5.	Mean >=3.5.
How likely would you use this website? (1-highly unlikely 5-highly likely)	Mean >=3.5.

If the success metrics are not met, we will conduct a design review on the aspect of the design that failed. For example, if users found the UI confusing, we will conduct a design review on the UI/UX.

Usability and Performance Evaluation

On top of JobRec being intuitive and useful for users, we want to also make sure that performance and usability is up to standards so that users do not get dissuaded by slow loading times and bad usability. During our user tests at our verification/validation milestones, we will measure performance and usability metrics, as defined below.

- Time to upload resume
- Time to generate job recommendations
- System response time
- Time for user to accomplish task

We define success metrics regarding these measures as follows:

Measure	**Success?**
Time to upload resume	< 2 seconds
Time to generate job recommendations	< 10 seconds
System response time	< 100ms
Time for user to accomplish task	< 2 minutes

If the metrics are not a success, we will conduct an investigation on what is affecting the performance, and where the confusion points are for the user experience.

Tools, Environments & Test Data Sets

Testing Tools:

- Pytest: backend unit and integration testing
- Postman: API testing
- Selenium: frontend and end-to-end workflow testing

Environments:

- Testing done in local development environment and staging environment that mirrors production
- Mock API responses and saved scraper outputs will be used when when external job sources are unavailable/inconsistent

Test Datasets:

- Sample resumes, job listings from scraper and third-party APIs, incomplete resumes (edge-case inputs), duplicate jobs, missing skill fields
- These datasets will help validate normal and failure scenarios

Version Control / Test Management:

- GitHub: source control and collaboration
- GitHub Actions: CI/CD and automated test reporting

Tool configurations will be standardized across the team. Pytest will be organized by the backend module, Postman will use saved collections for API routes, and Selenium will test major user flows such as resume upload and job recommendation display.

Test results will be archived through GitHub Action logs, saved test reports, and screenshots for failed UI tests. Important validation results will also be recorded in a shared report or spreadsheet for traceability.